In [3]:
```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
```

In [4]:
```
df=pd.read_csv('Heart Disease UCI.csv')
df.rename(columns={"condition": "target"}, inplace=True)
#Basic info
print(df.head())
print(df.info())
```

```
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  \
0   69    1   0       160   234    1        2      131      0      0.1      1
1   69    0   0       140   239    0        0      151      0      1.8      0
2   66    0   0       150   226    0        0      114      0      2.6      2
3   65    1   0       138   282    1        2      174      0      1.4      1
4   64    1   0       110   211    0        2      144      1      1.8      1

   ca  thal  target
0   1     0       0
1   2     0       0
2   0     0       0
3   1     0       1
4   0     0       0
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 297 entries, 0 to 296
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       297 non-null    int64
 1   sex       297 non-null    int64
 2   cp        297 non-null    int64
 3   trestbps  297 non-null    int64
 4   chol      297 non-null    int64
 5   fbs       297 non-null    int64
 6   restecg   297 non-null    int64
 7   thalach   297 non-null    int64
 8   exang     297 non-null    int64
 9   oldpeak   297 non-null    float64
 10  slope     297 non-null    int64
 11  ca        297 non-null    int64
 12  thal      297 non-null    int64
 13  target    297 non-null    int64
dtypes: float64(1), int64(13)
memory usage: 32.6 KB
None
```

In [5]:
```
print("\nColumn Names:\n", df.columns)

#Validate Data Types
print("\nData Types:\n", df.dtypes)
```

```
Column Names:
 Index(['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach',
       'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target'],
      dtype='object')

Data Types:
 age           int64
sex            int64
cp             int64
trestbps       int64
chol           int64
fbs            int64
restecg        int64
thalach        int64
exang          int64
oldpeak      float64
slope          int64
ca             int64
thal           int64
target         int64
dtype: object
```

In [7]:

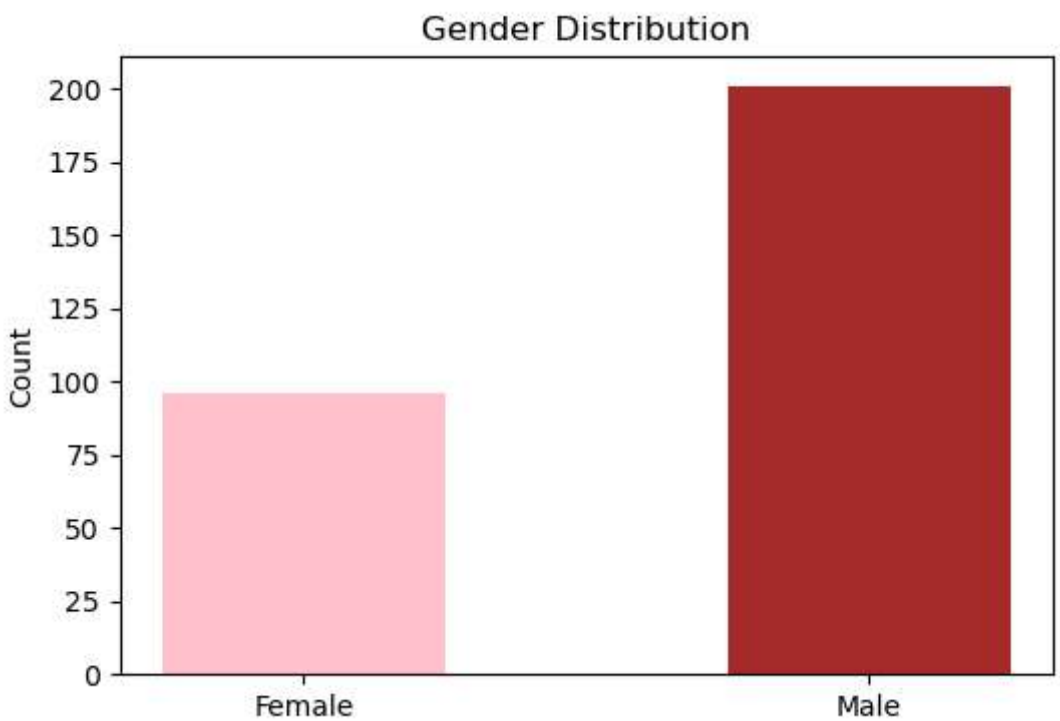
```python
print('Shape:', df.shape)
print('\nColumn names and dtypes:')
print(df.dtypes)

#checking missing values
print('\nMissing values per column:')
print(df.isnull().sum())

#distribution
print('\ncondition distribution:')
print(df['target'].value_counts(dropna=False))

#summary statistics
print('\nDescriptive statistics:')
display(df.describe())
```

```
Shape: (297, 14)

Column names and dtypes:
age           int64
sex           int64
cp            int64
trestbps      int64
chol          int64
fbs           int64
restecg       int64
thalach       int64
exang         int64
oldpeak     float64
slope         int64
ca            int64
thal          int64
target        int64
dtype: object

Missing values per column:
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          0
thal        0
target      0
dtype: int64

condition distribution:
target
0    160
1    137
Name: count, dtype: int64

Descriptive statistics:
```

	age	sex	cp	trestbps	chol	fbs	restecg
count	297.000000	297.000000	297.000000	297.000000	297.000000	297.000000	297.000000
mean	54.542088	0.676768	2.158249	131.693603	247.350168	0.144781	0.996633
std	9.049736	0.468500	0.964859	17.762806	51.997583	0.352474	0.994914
min	29.000000	0.000000	0.000000	94.000000	126.000000	0.000000	0.000000
25%	48.000000	0.000000	2.000000	120.000000	211.000000	0.000000	0.000000
50%	56.000000	1.000000	2.000000	130.000000	243.000000	0.000000	1.000000
75%	61.000000	1.000000	3.000000	140.000000	276.000000	0.000000	2.000000
max	77.000000	1.000000	3.000000	200.000000	564.000000	1.000000	2.000000

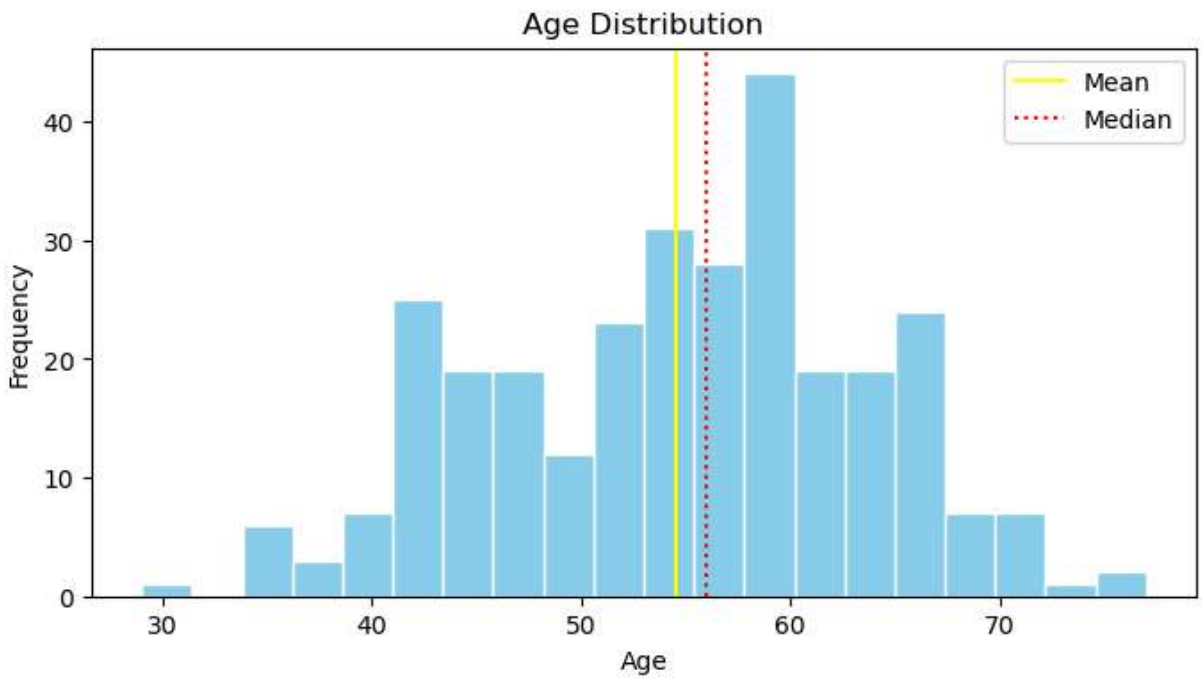
In [7]:
```
# 2. Gender Distribution Analysis
gender_counts = df["sex"].value_counts().sort_index()
gender_pct = np.round((gender_counts.values / len(df)) * 100, 2)

print("Female:", gender_counts[0], "->", gender_pct[0], "%")
print("Male  :", gender_counts[1], "->", gender_pct[1], "%")

plt.figure(figsize=(6, 4))
plt.bar(["Female", "Male"], gender_counts.values, color=["pink", "brown"], width=0.
plt.title("Gender Distribution")
plt.ylabel("Count")
plt.show()
```

```
Female: 96 -> 32.32 %
Male  : 201 -> 67.68 %
```



In [10]:
```
# 3. Age Analysis
print("Minimum Age:", df["age"].min())
print("Maximum Age:", df["age"].max())
print("Mean Age   :", df["age"].mean())
print("Median Age :", df["age"].median())

plt.figure(figsize=(8, 4))
plt.hist(df["age"], bins=20, color="skyblue", edgecolor="white")
plt.axvline(df["age"].mean(),   color="yellow", linestyle="-", label="Mean")
plt.axvline(df["age"].median(), color="red",   linestyle=":",  label="Median")
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.legend()
plt.show()
```

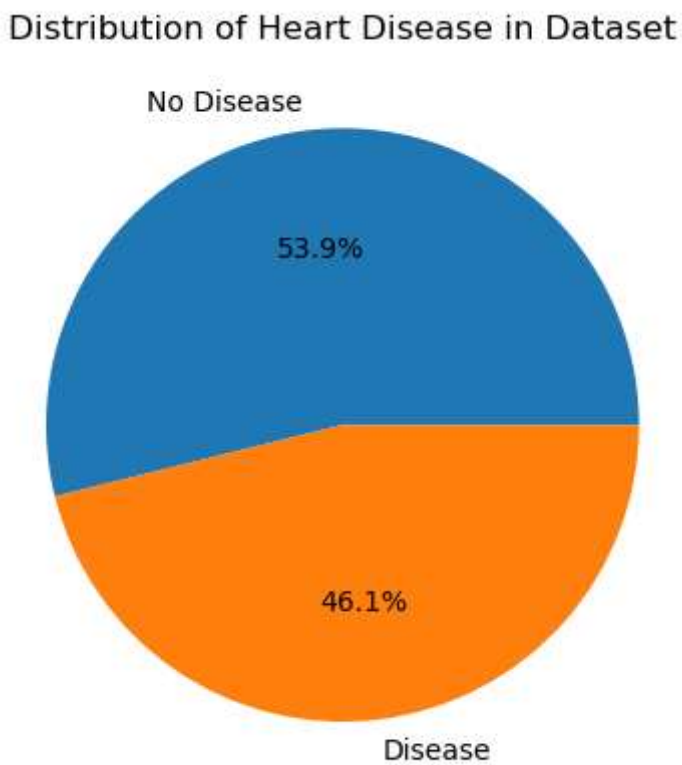
```
Minimum Age: 29
Maximum Age: 77
Mean Age   : 54.54208754208754
Median Age : 56.0
```



In [32]:
```
# 4.Count number of patients in each class
target_counts = df['target'].value_counts()

print("Patient Count by Heart Disease Status:")
print("No Disease (0):", target_counts.get(0, 0))
print("Disease (1):", target_counts.get(1, 0))

total_patients = len(df)
disease_percentage = (target_counts.get(1, 0) / total_patients) * 100
no_disease_percentage = (target_counts.get(0, 0) / total_patients) * 100

print(f"Percentage of patients with heart disease: {disease_percentage:.2f}%")
print(f"Percentage of patients without heart disease: {no_disease_percentage:.2f}%"
```

```
labels = ['No Disease', 'Disease']
sizes = [target_counts.get(0, 0), target_counts.get(1, 0)]

plt.figure()
plt.pie(sizes, labels=labels, autopct='%1.1f%%')
plt.title("Distribution of Heart Disease in Dataset")
plt.show()
```

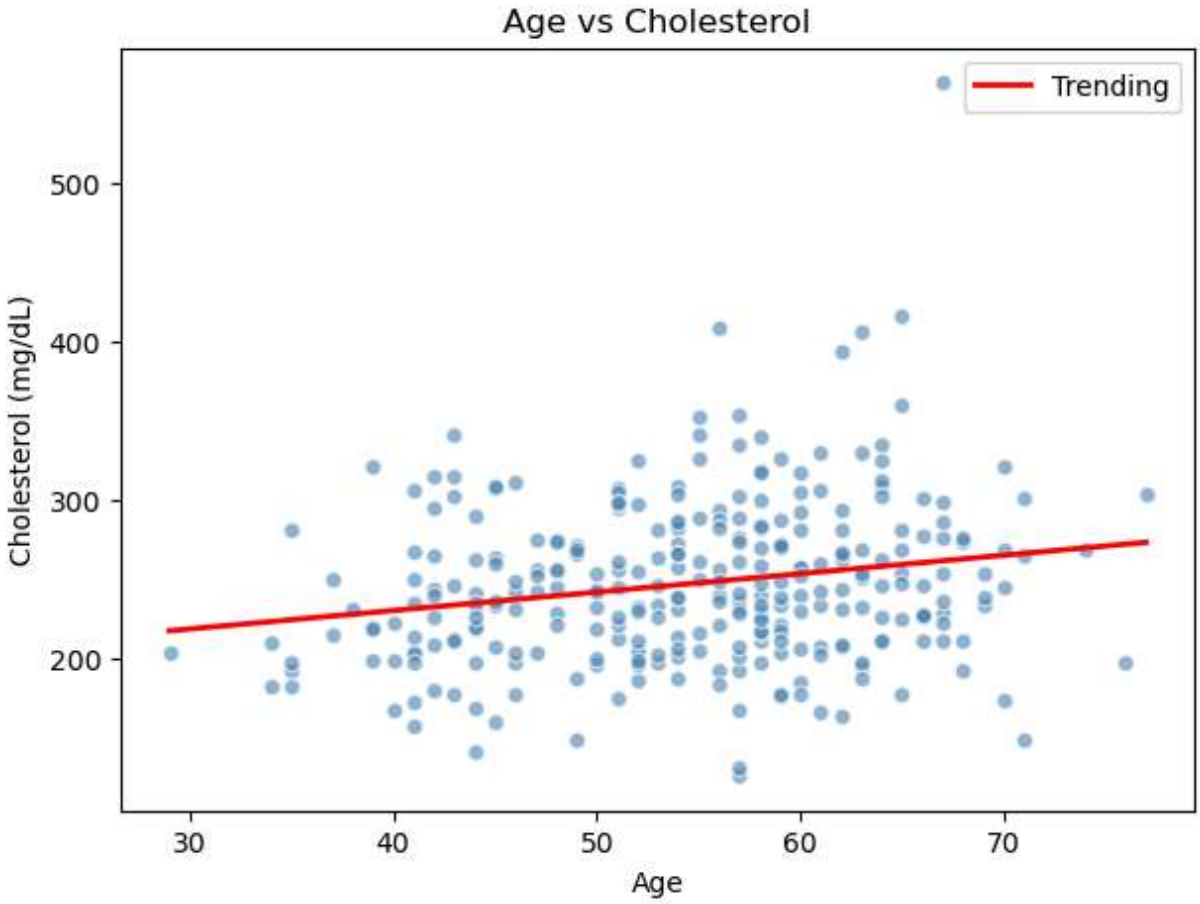
```
Patient Count by Heart Disease Status:
No Disease (0): 160
Disease (1): 137
Percentage of patients with heart disease: 46.13%
Percentage of patients without heart disease: 53.87%
```



In [12]:

```
# 5. Correlation Between Age and Cholesterol
corr_matrix = df[["age", "chol"]].corr()
print("Correlation using df.corr():")
print(corr_matrix)
print("\nPearson r:", round(corr_matrix.loc["age", "chol"], 4))

m, b = np.polyfit(df["age"], df["chol"], 1)

plt.figure(figsize=(7, 5))
plt.scatter(df["age"], df["chol"], alpha=0.6, color="steelblue", edgecolors="white"
plt.plot(sorted(df["age"]), m * np.sort(df["age"]) + b, color="red", linewidth=2, l
plt.title("Age vs Cholesterol")
plt.xlabel("Age")
plt.ylabel("Cholesterol (mg/dL)")
plt.legend()
plt.show()

print("Interpretation: Weak positive correlation — cholesterol slightly increases w
```

```
Correlation using df.corr():
           age      chol
age   1.000000  0.202644
chol  0.202644  1.000000

Pearson r: 0.2026
```

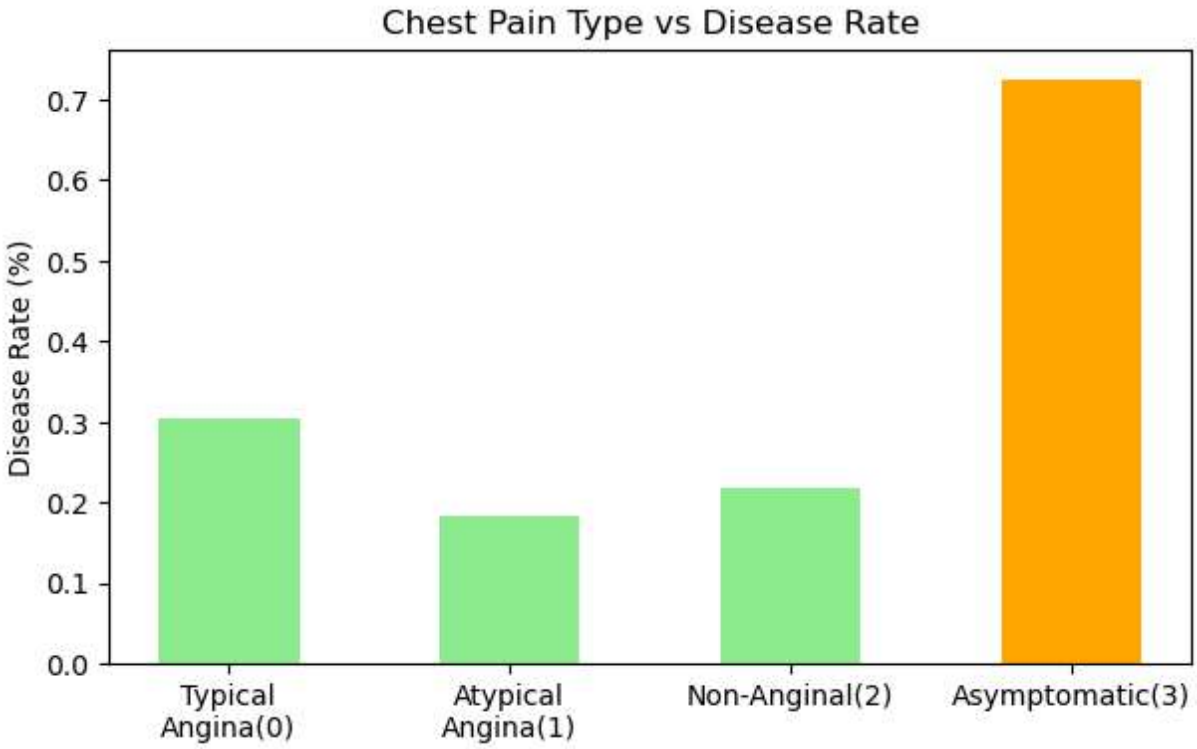



```
Interpretation: Weak positive correlation — cholesterol slightly increases with age
```

In [24]:

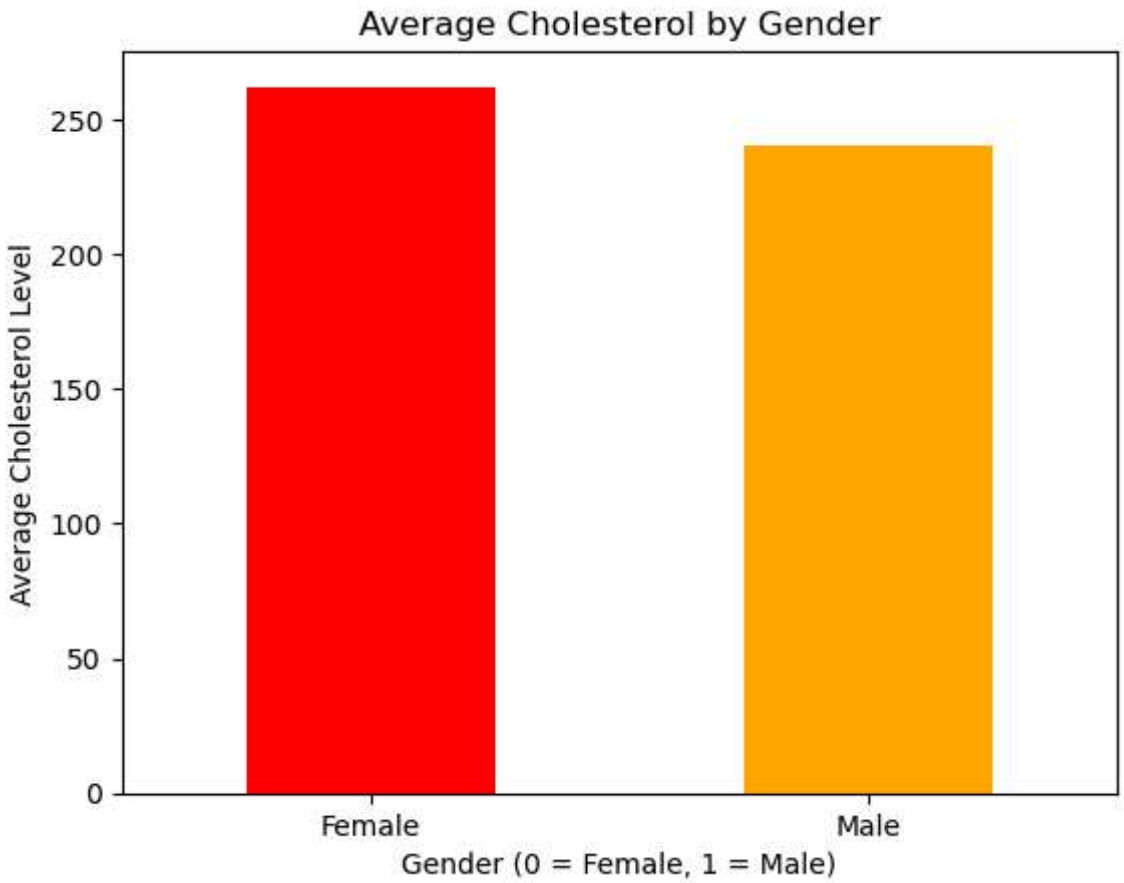
```
# 6. Chest Pain Type vs Disease
cp_disease = df.groupby("cp")["target"].mean()
print("Disease Rate by Chest Pain Type:")
print(cp_disease)
print("\nMost risky cp type:", cp_disease.idxmax(), "->", round(cp_disease.max(), 1

colors = ["orange" if i == cp_disease.idxmax() else "lightgreen" for i in cp_diseas

plt.figure(figsize=(7, 4))
plt.bar(["Typical\nAngina(0)", "Atypical\nAngina(1)", "Non-Anginal(2)", "Asymptomat
        cp_disease.values, color=colors, width=0.5)
plt.title("Chest Pain Type vs Disease Rate")
plt.ylabel("Disease Rate (%)")
plt.show()
```

```
Disease Rate by Chest Pain Type:
cp
0    0.304348
1    0.183673
2    0.216867
3    0.725352
Name: target, dtype: float64

Most risky cp type: 3 -> 0.7 %
```



In [33]:
```python
#7. Calculate average cholesterol by gender
avg_chol_gender = df.groupby('sex')['chol'].mean()

print("Average Cholesterol by Gender:")
print(avg_chol_gender)

avg_chol_gender.plot(kind='bar',color=['red','orange'],width=0.5)
plt.title("Average Cholesterol by Gender")
plt.xlabel("Gender (0 = Female, 1 = Male)")
plt.ylabel("Average Cholesterol Level")
plt.xticks([0, 1], ["Female", "Male"], rotation=0)
plt.show()
```

```
Average Cholesterol by Gender:
sex
0    262.229167
1    240.243781
Name: chol, dtype: float64
```



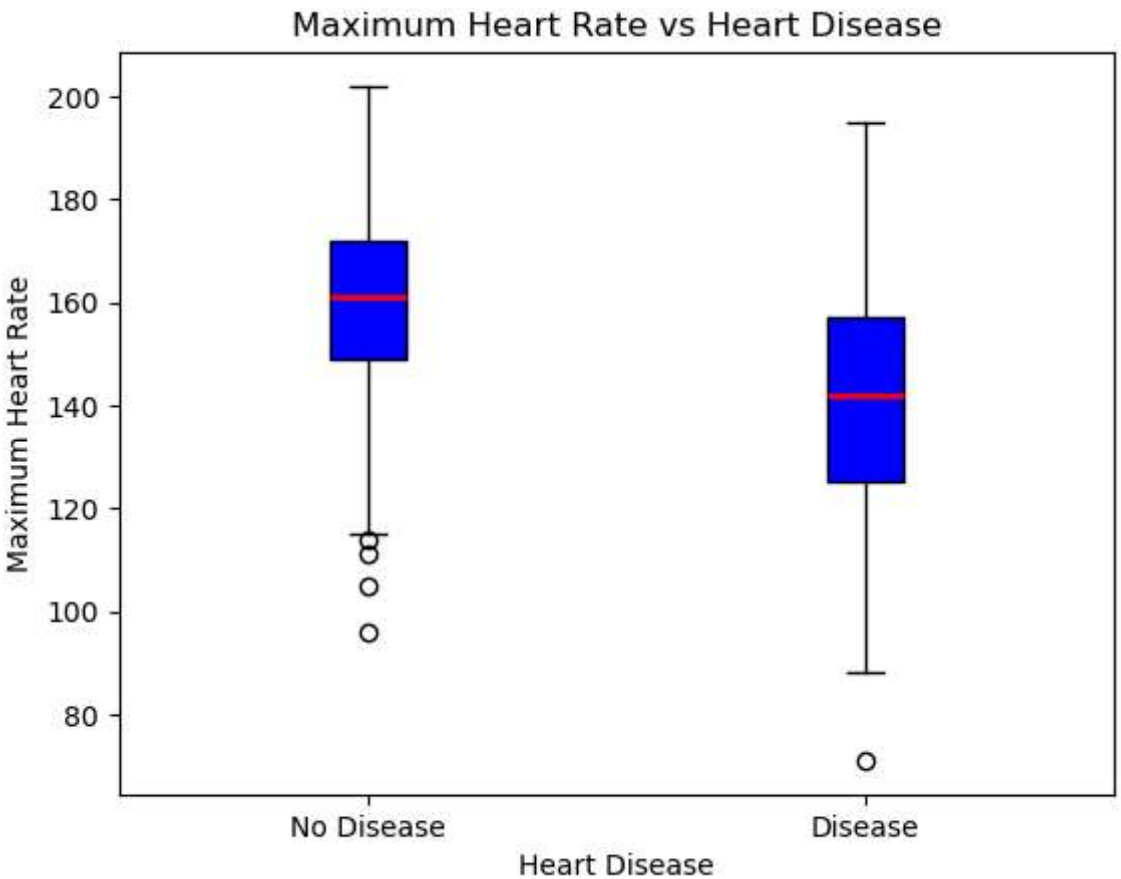
In [14]:
```
# 8. Resting Blood Pressure Analysis
avg_bp  = df["trestbps"].mean()
high_bp = df[df["trestbps"] > 140]

print("Average BP                 :", round(avg_bp, 2))
print("Patients with BP > 140     :", len(high_bp))
print("Disease % in high BP group :", round(high_bp["target"].mean() * 100, 1), "%"
print("Overall disease %          :", round(df["target"].mean() * 100, 1), "%")
print("Conclusion: High BP patients have higher disease rate than average")
```

```
Average BP                 : 131.69
Patients with BP > 140     : 66
Disease % in high BP group : 59.1 %
Overall disease %          : 46.1 %
Conclusion: High BP patients have higher disease rate than average
```

In [39]:
```
#9. Average max heart rate by disease status
avg_thalach = df.groupby('target')['thalach'].mean()

print("Average Maximum Heart Rate by Disease Status:")
print("No Disease (0):", round(avg_thalach.get(0, 0), 2))
print("Disease (1):", round(avg_thalach.get(1, 0), 2))

thalach_no = df[df['target'] == 0]['thalach']
thalach_yes = df[df['target'] == 1]['thalach']

plt.boxplot([thalach_no, thalach_yes],
            tick_labels=["No Disease", "Disease"],
            patch_artist=True,
```

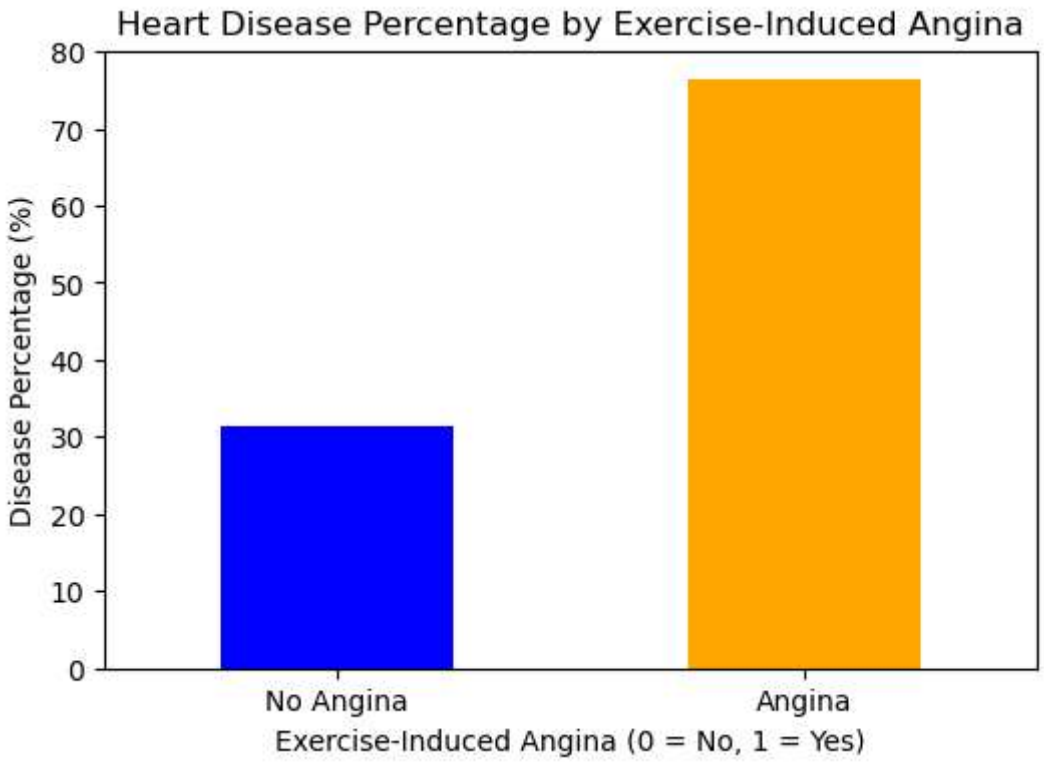
```
            boxprops=dict(facecolor="blue"),
            medianprops=dict(color="red", linewidth=2))
plt.title("Maximum Heart Rate vs Heart Disease")
plt.xlabel("Heart Disease")
plt.ylabel("Maximum Heart Rate")
plt.show()
```

```
Average Maximum Heart Rate by Disease Status:
No Disease (0): 158.58
Disease (1): 139.11
```



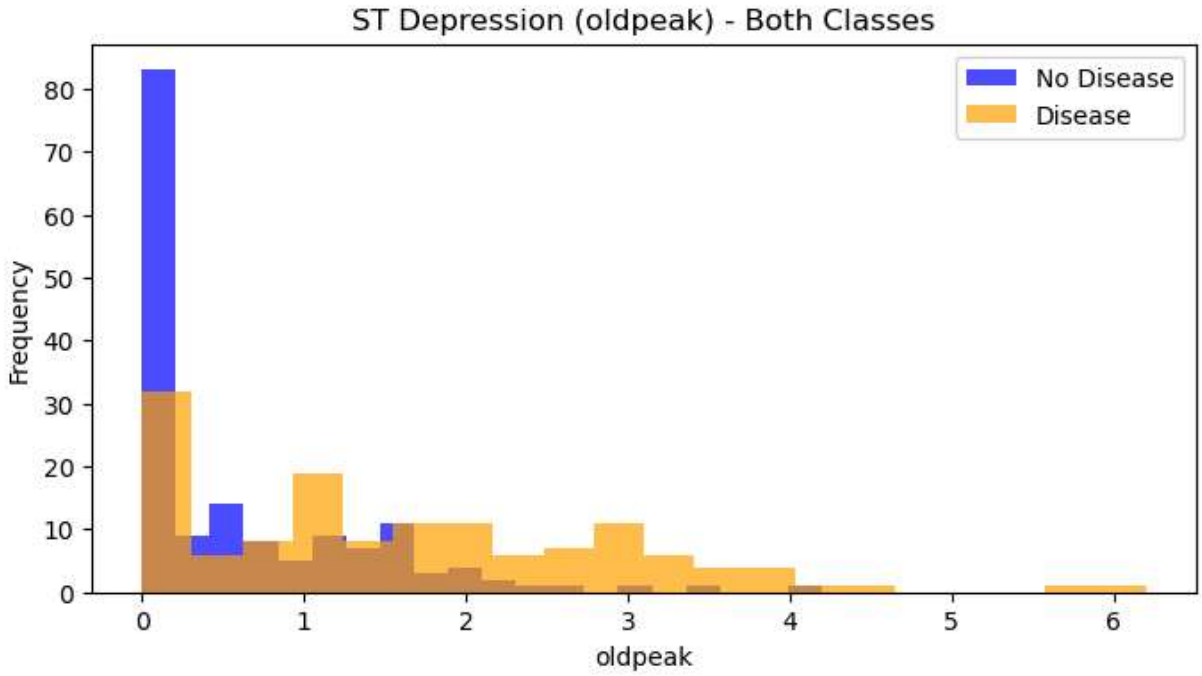
In [44]:
```
# 10. Exercise Induced Angina Impact
exang_disease_rate = df.groupby('exang')['target'].mean() * 100

print("Disease Percentage by Exercise-Induced Angina:")
print("No Angina (exang = 0):", round(exang_disease_rate.get(0, 0), 2), "%")
print("Angina Present (exang = 1):", round(exang_disease_rate.get(1, 0), 2), "%")

plt.figure(figsize=(6,4))
exang_disease_rate.plot(kind='bar',color=['blue','orange'])
plt.title("Heart Disease Percentage by Exercise-Induced Angina")
plt.xlabel("Exercise-Induced Angina (0 = No, 1 = Yes)")
plt.ylabel("Disease Percentage (%)")
plt.xticks([0, 1], ["No Angina", "Angina"], rotation=0)
plt.show()
```

```
Disease Percentage by Exercise-Induced Angina:
No Angina (exang = 0): 31.5 %
Angina Present (exang = 1): 76.29 %
```



In [16]:
```
# 11. ST Depression (oldpeak) Analysis
op_mean = df.groupby("target")["oldpeak"].mean().round(3)
print("Mean oldpeak (No Disease):", op_mean[0])
print("Mean oldpeak (Disease)   :", op_mean[1])

plt.figure(figsize=(8, 4))
plt.hist(df[df["target"] == 0]["oldpeak"], bins=20, alpha=0.7, color="blue", label=
plt.hist(df[df["target"] == 1]["oldpeak"], bins=20, alpha=0.7, color="orange",    l
plt.title("ST Depression (oldpeak) - Both Classes")
plt.xlabel("oldpeak")
plt.ylabel("Frequency")
plt.legend()
plt.show()

print("Trend: Higher oldpeak is strongly associated with heart disease")
```

```
Mean oldpeak (No Disease): 0.599
Mean oldpeak (Disease)   : 1.589
```

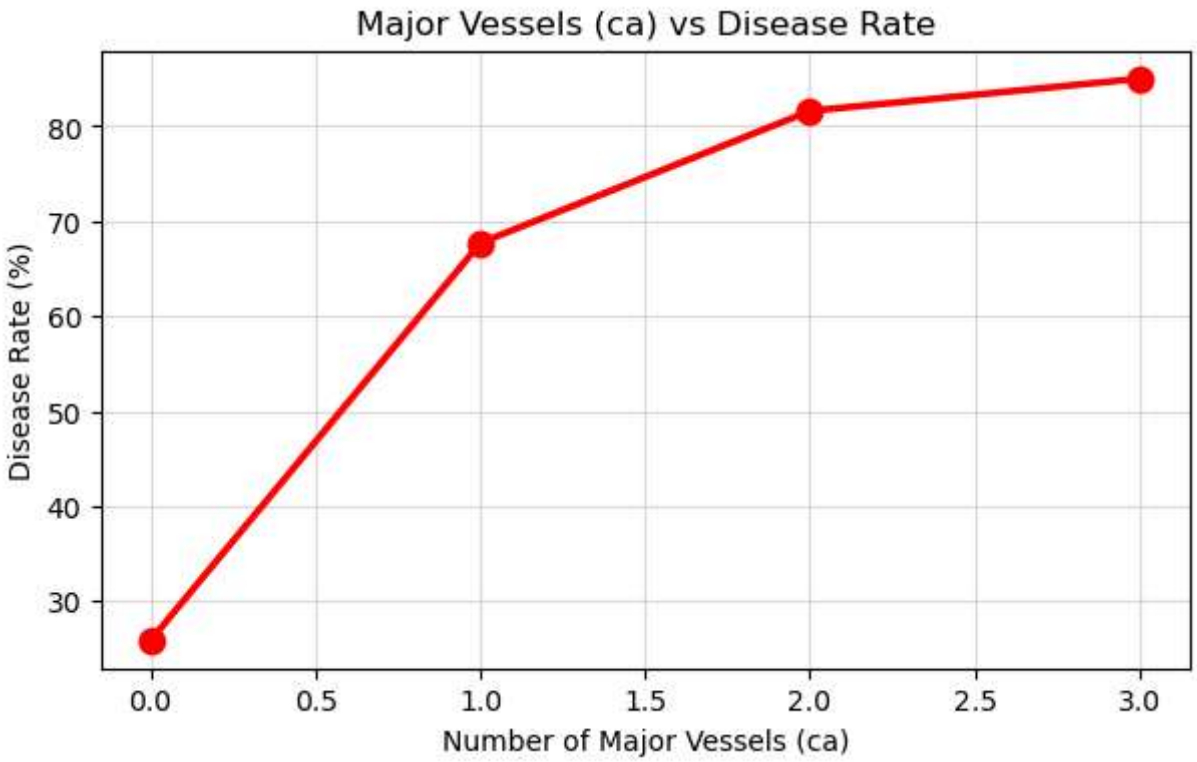



```
Trend: Higher oldpeak is strongly associated with heart disease
```

In [17]:

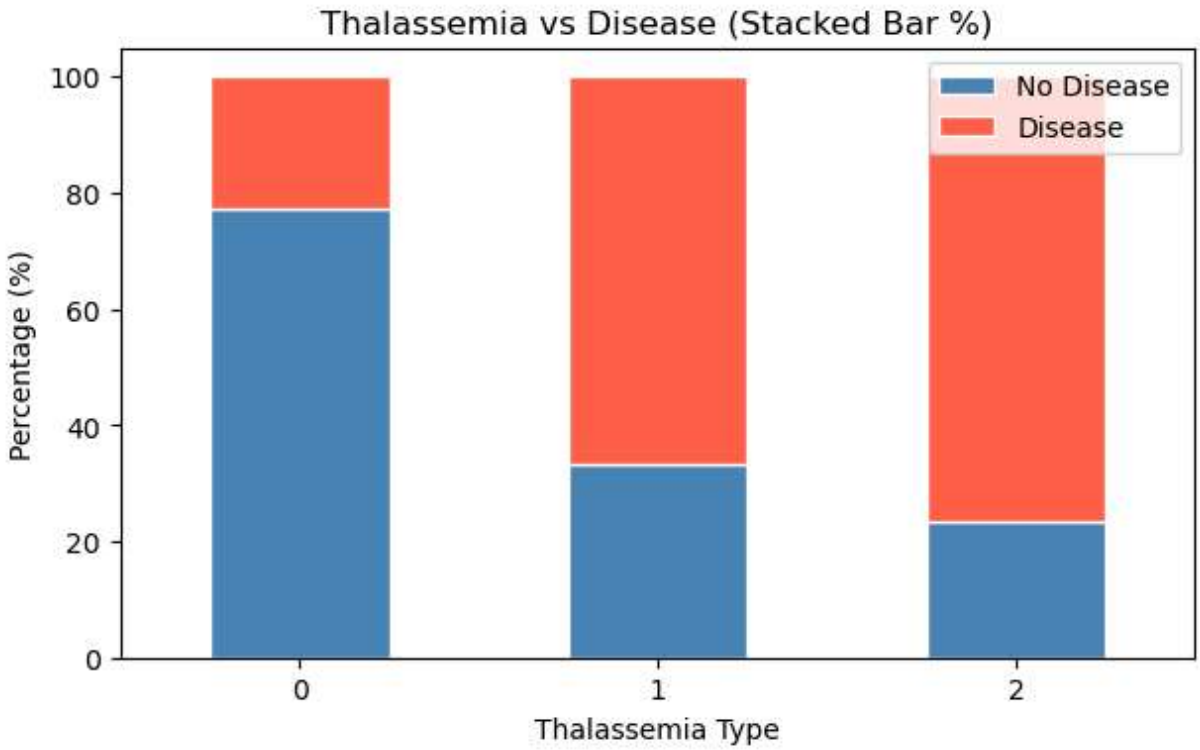
```python
# 12. Number of Major Vessels (ca) Impact
ca_disease = df.groupby("ca")["target"].mean().round(3)
print("Disease Probability by Major Vessels:")
print(ca_disease)

plt.figure(figsize=(7, 4))
plt.plot(ca_disease.index, ca_disease.values * 100,
         marker="o", color="red", linewidth=2.5, markersize=9)
plt.title("Major Vessels (ca) vs Disease Rate")
plt.xlabel("Number of Major Vessels (ca)")
plt.ylabel("Disease Rate (%)")
plt.grid(alpha=0.4)
plt.show()
```

```
Disease Probability by Major Vessels:
ca
0    0.259
1    0.677
2    0.816
3    0.850
Name: target, dtype: float64
```



In [19]:
```
# 13. Thalassemia vs Disease
thal_ct  = pd.crosstab(df["thal"], df["target"])
thal_pct = thal_ct.div(thal_ct.sum(axis=1), axis=0) * 100

print("Cross-tabulation (%):")
print(thal_pct.round(1))

thal_pct.plot(kind="bar", stacked=True, figsize=(7, 4),
              color=["steelblue", "tomato"], edgecolor="white")
plt.title("Thalassemia vs Disease (Stacked Bar %)")
plt.xlabel("Thalassemia Type")
plt.ylabel("Percentage (%)")
plt.legend(["No Disease", "Disease"])
plt.xticks(rotation=0)
plt.show()
```

```
Cross-tabulation (%):
target     0     1
thal
0       77.4  22.6
1       33.3  66.7
2       23.5  76.5
```



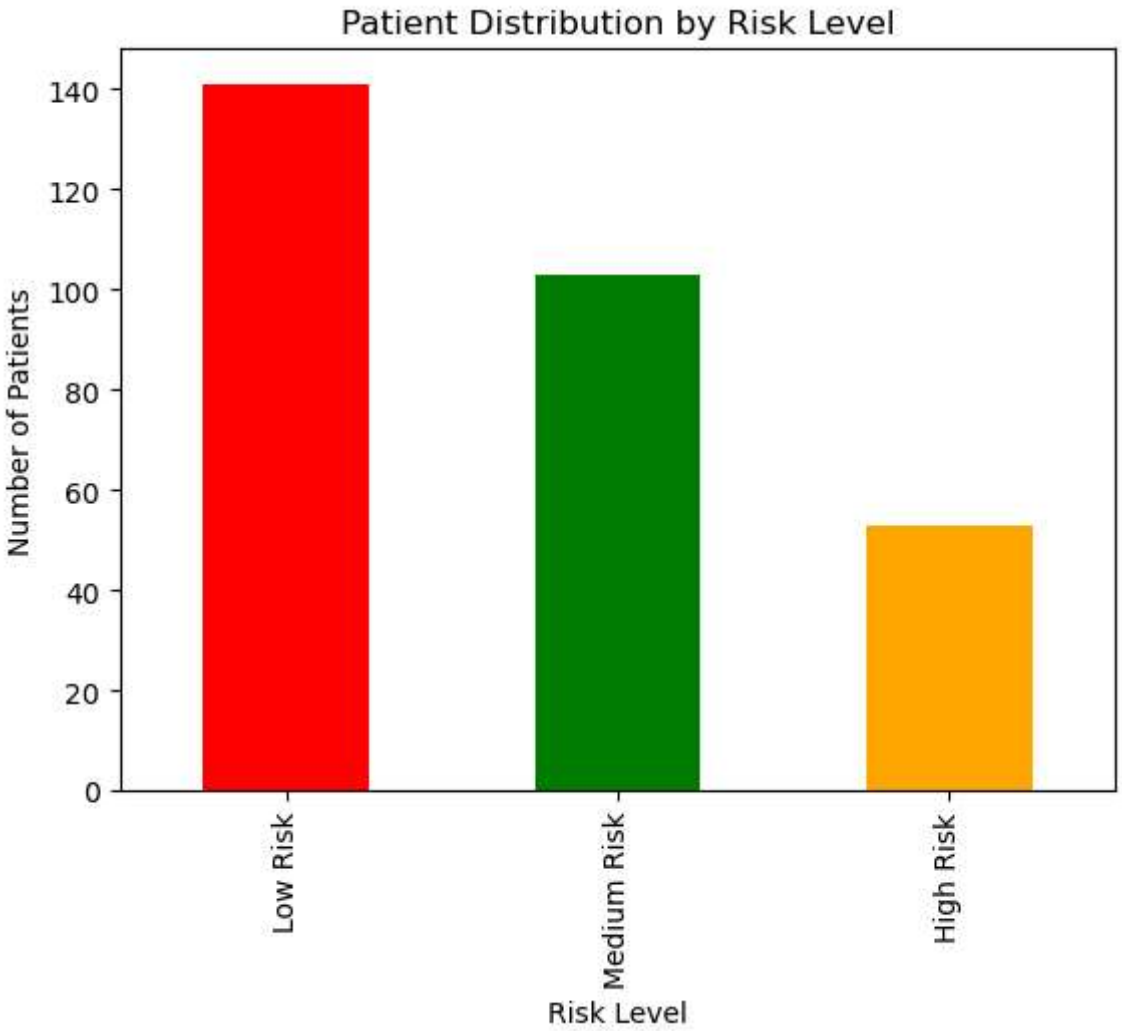
In [20]:
```
# 14. Multi-Factor Risk Analysis using NumPy
arr        = df.values
cols       = list(df.columns)
age_idx    = cols.index("age")
chol_idx   = cols.index("chol")
bp_idx     = cols.index("trestbps")
target_idx = cols.index("target")

mask = (
    (arr[:, age_idx].astype(float)  > 50)  &
    (arr[:, chol_idx].astype(float) > 240) &
    (arr[:, bp_idx].astype(float)   > 140)
)

high_risk   = arr[mask]
disease_pct = np.round(high_risk[:, target_idx].astype(float).sum() / len(high_risk

print("Patients with Age>50, Chol>240, BP>140:", len(high_risk))
print("Disease % in this high-risk group      :", disease_pct, "%")
```

```
Patients with Age>50, Chol>240, BP>140: 33
Disease % in this high-risk group      : 66.67 %
```

In [18]:
```
#15. Create Risk Score (Custom Analysis)

# Create custom risk score
df['risk_score'] = (df['chol'] / 200) + (df['trestbps'] / 120) + (df['oldpeak'])
# Define risk categories based on risk_score
def classify_risk(score):
    if score < 3:
        return "Low Risk"
    elif score < 4.5:
        return "Medium Risk"
```

```
    else:
        return "High Risk"

df['risk_level'] = df['risk_score'].apply(classify_risk)

# Check distribution
print(df['risk_level'].value_counts())
df['risk_level'].value_counts().plot(kind='bar',color=['red','green','orange'])
plt.title("Patient Distribution by Risk Level")
plt.xlabel("Risk Level")
plt.ylabel("Number of Patients")
plt.show()
```

```
risk_level
Low Risk       141
Medium Risk    103
High Risk       53
Name: count, dtype: int64
```



In [ ]:

Heart Disease Analysis

1. Introduction

Heart disease is one of the major health problems across the world. Hospitals and healthcare providers need to identify people who are at high risk as early as possible so that proper treatment and monitoring can be provided. In this project, patient clinical data is analyzed to understand which factors are linked to heart disease and how this information can support doctors and hospitals in making better decisions.

2. Objective

The main objective of this project is to analyze patient health data and identify important factors that influence the presence of heart disease. The project aims to:

- Understand the relationship between patient characteristics and heart disease
- Identify high-risk patient groups
- Provide simple, data-driven insights that can support early screening and preventive care.

3. Dataset Description

The dataset contains patient information such as age, gender, cholesterol level, resting blood pressure, maximum heart rate, chest pain type, and exercise-induced angina. The target column condition indicates whether a patient has heart disease (1) or not (0).
The dataset was cleaned and validated before performing analysis to ensure accuracy and reliability of results.

4. Methodology

The analysis was carried out using the following steps:

- **Data Cleaning:**
  The dataset was checked for missing values, correct data types, and outliers to ensure data quality.
- **Exploratory Data Analysis (EDA):**
  Various visualizations such as histograms, bar charts, and scatter plots were used to understand the distribution of data and compare patients with and without heart disease.
- **Statistical Analysis:**
  Basic statistical measures such as mean, median, and standard deviation were

calculated. Correlation analysis was performed to identify features strongly related to heart disease.

- **Risk Segmentation:**
  Patients were grouped into Low Risk, Medium Risk, and High Risk categories based on age, cholesterol, blood pressure, and ST depression (oldpeak) to identify patients who require closer monitoring.

5. Key Insights

1. Does cholesterol strongly impact heart disease?

Cholesterol has a moderate impact, but it is not the strongest factor on its own.

Patients with heart disease generally show slightly higher cholesterol levels, but there is significant overlap between diseased and non-diseased groups. This indicates that cholesterol alone is not a strong predictor and should be considered along with other factors such as age and blood pressure.

2. Is the male population more vulnerable?

Yes, male patients are more vulnerable to heart disease.

Gender-based analysis shows that heart disease is more common among male patients compared to female patients. This suggests a higher vulnerability in the male population and highlights the need for targeted screening and preventive care for males.

3. Does exercise-induced angina significantly increase risk?

Yes, exercise-induced angina significantly increases the risk of heart disease.

Patients who experience chest pain during exercise show a much higher prevalence of heart disease. This makes exercise-induced angina one of the strongest clinical warning signs in the dataset.

4. Which feature has the strongest correlation with disease?

ST depression (oldpeak) and number of major vessels (ca) show the strongest correlation with heart disease.
Maximum heart rate (thalach) shows a strong negative correlation.

Correlation analysis indicates that oldpeak and major vessel count are the most strongly associated features with heart disease presence, while higher maximum heart rate is linked with lower disease likelihood

6. Risk Group Analysis

Patients were categorized into Low Risk, Medium Risk, and High Risk groups based on key clinical parameters such as age, cholesterol, blood pressure, and oldpeak. The high-risk group showed a much higher presence of heart disease compared to the low-risk group. This type of grouping can help hospitals focus more attention on patients who are more likely to develop serious heart conditions.

7. Conclusion

This project shows that simple data analysis can help to identify people who are more likely to have heart disease. These insights help to find problems early, to keep track pf patients health and to plan healthcare services more efficiently.